CSCI 350:
Fundamentals of Operating Systems

Concurrency, Threads

Prof. Tatyana Ryutov

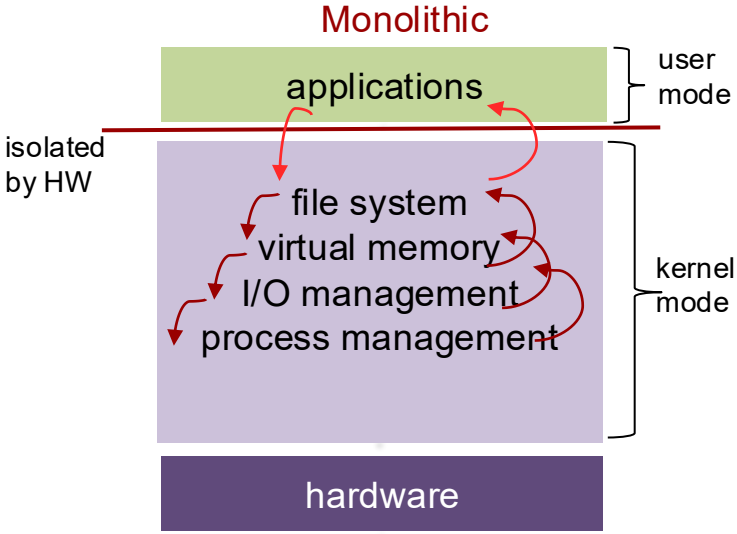
This content is protected and may not be shared, uploaded, or distributed.

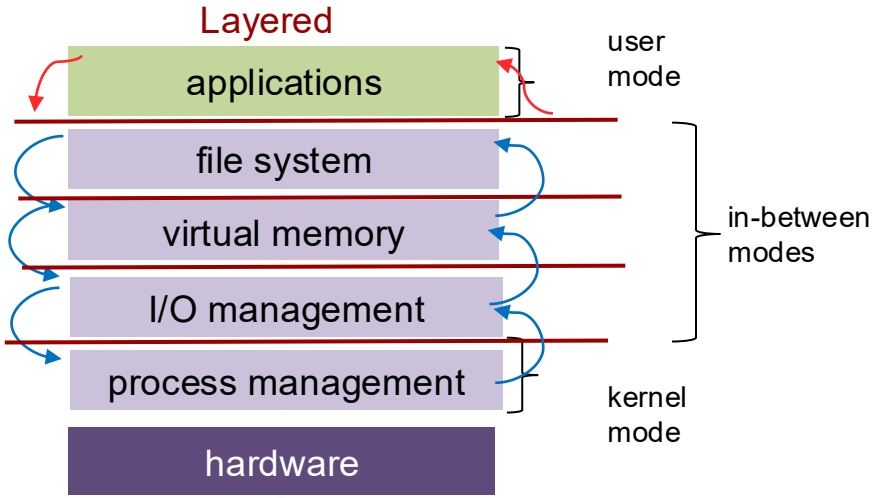
Outline

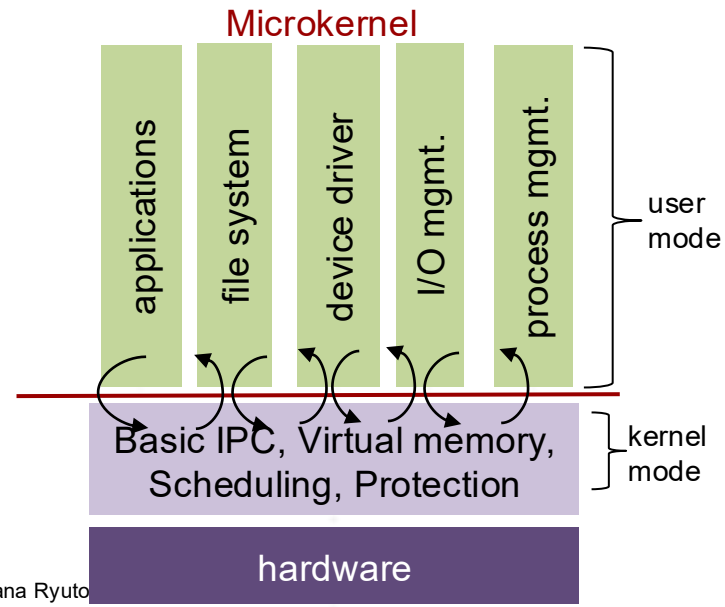
- Review
- Concurrency
- Thread abstraction
- Thread operations
- Implementing threads
- Synchronization intro

©Tatyana Ryutov

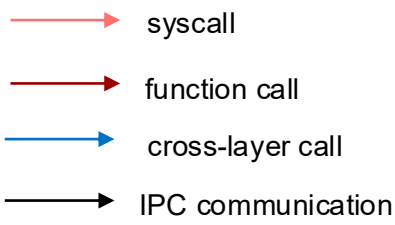
Review: Kernel Organization Approaches and Tradeoffs







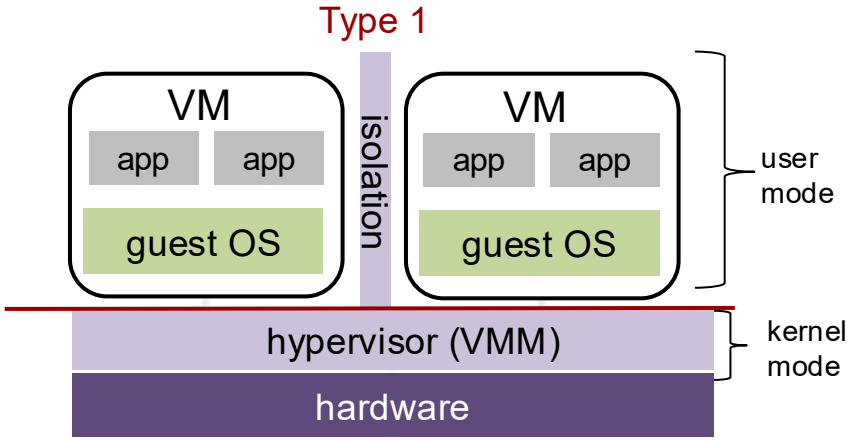
	Performance	Reliability Security	Flexibility Comprehensibility
Monolithic	best	worst	worst
Layered	bad	best	good
Microkernel	bad	good	best

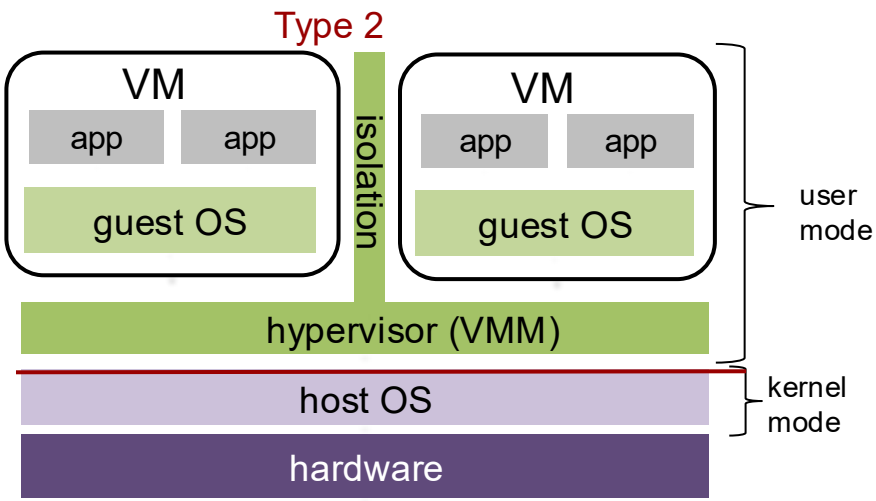


©Tatyana Ryuto

USC Viterbi School of Engineering

Review: Hypervisor Types





	Performance	Reliability Security	Flexibility Comprehensibility
Type 1	good	good	bad
Type 2	bad	bad	good

- Why is the hosted VM with more layers of abstraction less secure than the native VM?

Mac OS ≈ 80? million lines of code

Linux ≈ 15 million lines of code

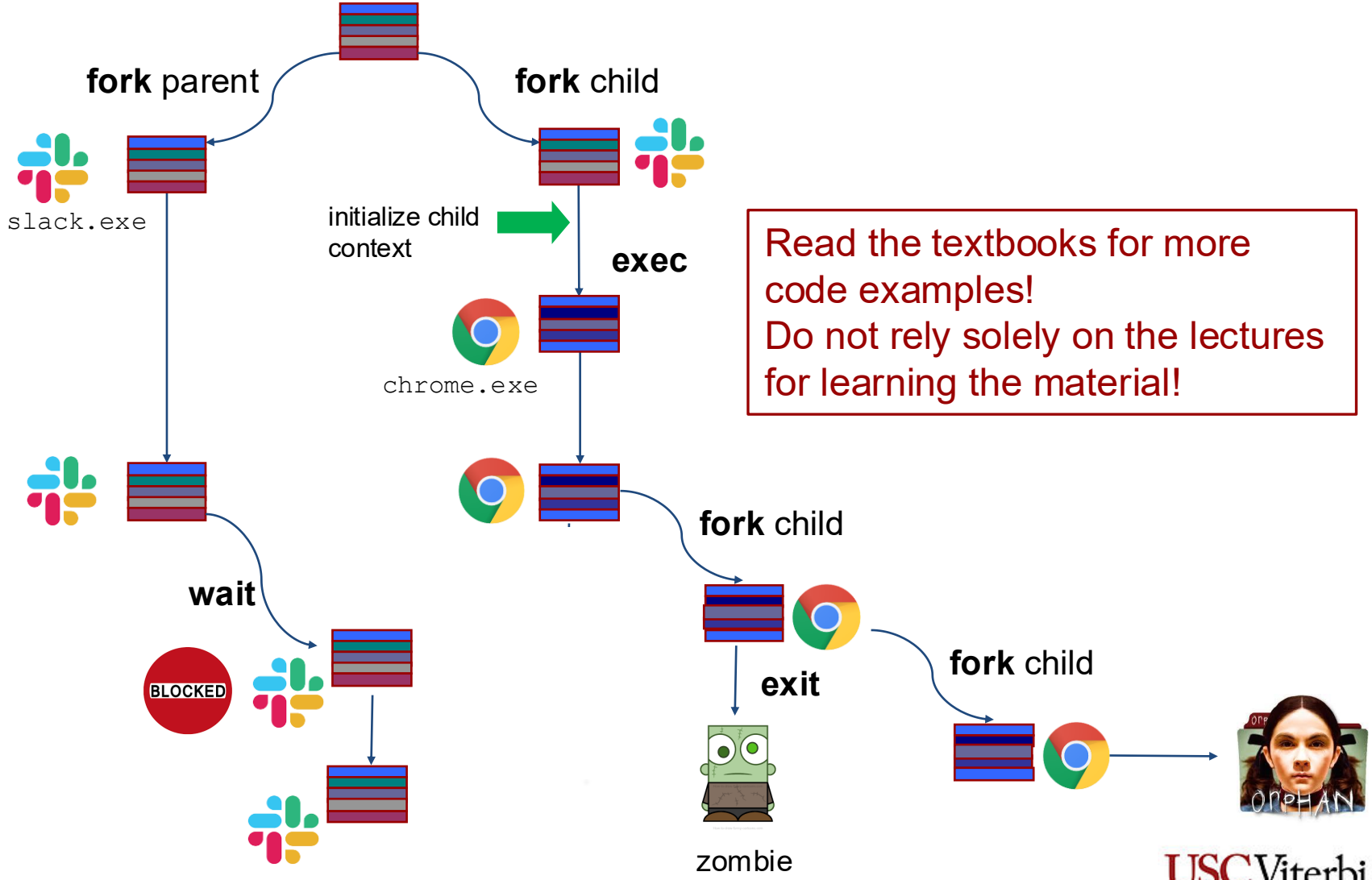
Xen Server™ ≈70K lines of code

KVM ≈10K lines of code

©Tatyana Ryutov

USC Viterbi School of Engineering

Review: Unix `fork/exec/exit/wait`



Read the textbooks for more code examples!
Do not rely solely on the lectures for learning the material!

USC Viterbi School of Engineering

©Tatyana Ryutov

Example: Shell

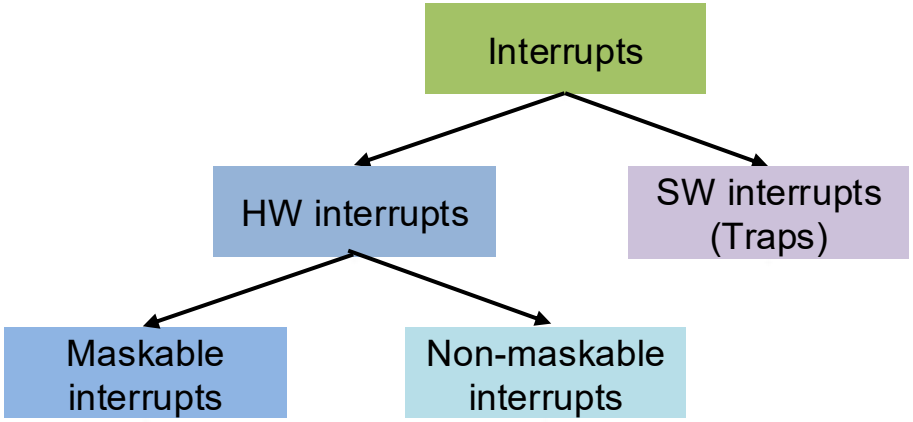
- A shell is a job control system
  - Allows programmer to create and manage a set of programs to do some task
  - Windows, MacOS, Linux all have shells

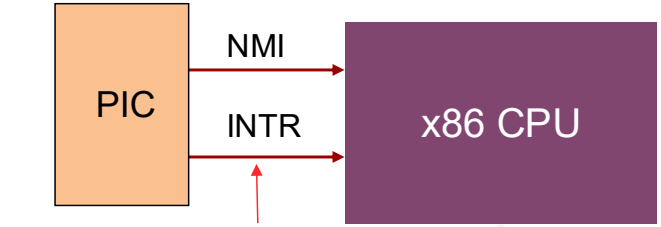
```
char *prog, **args;
int child_pid;

// Read and parse the input a line at a time

while (readAndParseCmdLine(&prog, &args)) {

   child_pid = fork();       // create a child process

    if (child_pid == 0) {
       exec(prog, args);     // I'm the child process, run "prog"
    } else {
       wait(child_pid);      // I'm parent, wait for child
    }
}
```

©Tatyana Ryutov

Review: Interrupts and Traps

- Interrupt is a hardware mechanism that enables CPU to detect that a device needs its attention
- Software Interrupts (Traps) – synchronous
  - Generated by SW with a special assembly instruction, e.g., system call, math error, page fault
- Hardware Interrupts – asynchronous, spawned by event external to CPU
  - Generated by devices when request is complete or an event that requires CPU attention happens
  - Maskable
    - Can be disabled or ignored by the CPU instructions, e.g., not delivered on the INTR line
  - Non-maskable
    - Cannot be disabled by the CPU instructions, e.g., always invoked by NMI line





processor recognizes interrupt only if interrupt flag `IF` is set

©Tatyana Ryutov

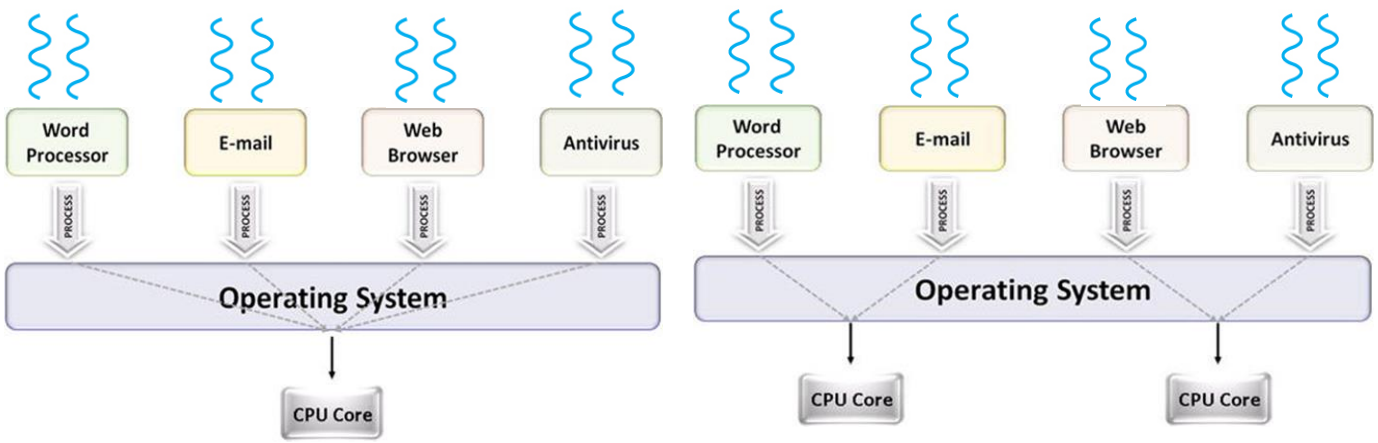
Outline

- Review
- Concurrency
- Thread abstraction
- Thread operations
- Implementing threads
- Synchronization intro

©Tatyana Ryutov

Concurrency in OS

- Concurrency is the interleaving of processes (or threads) in time to give the appearance of simultaneous execution
  - Differs from parallelism, which offers genuine simultaneous execution
- Process management in OS can be classified as:
  - Multiprogramming: interleaved execution of two or more processes by a single CPU computer system
  - Multiprocessing: simultaneous execution of two or more process by a computer having more than one CPU

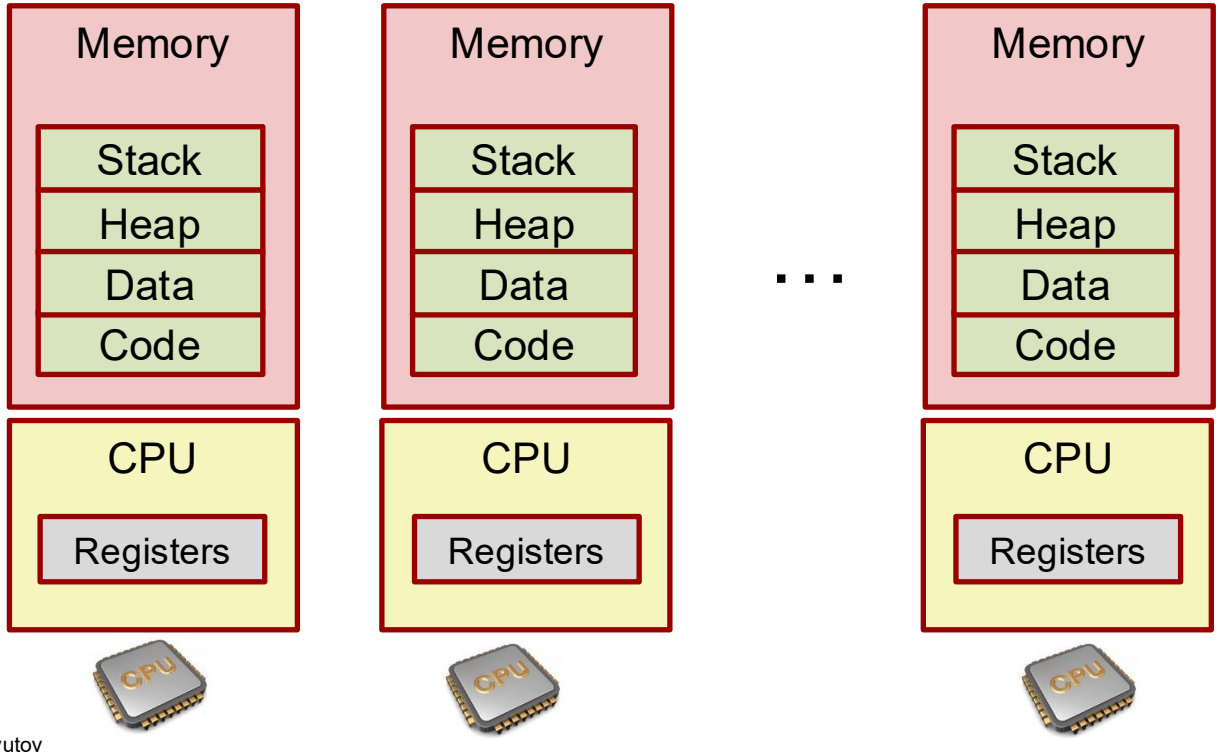


©Tatyana Ryutov

USC Viterbi School of Engineering

Concurrency: The Illusion

- Computer runs many processes simultaneously
  - Applications for one or more users
    - Web browsers, email clients, editors, …
  - Background tasks
    - Monitoring network & I/O devices

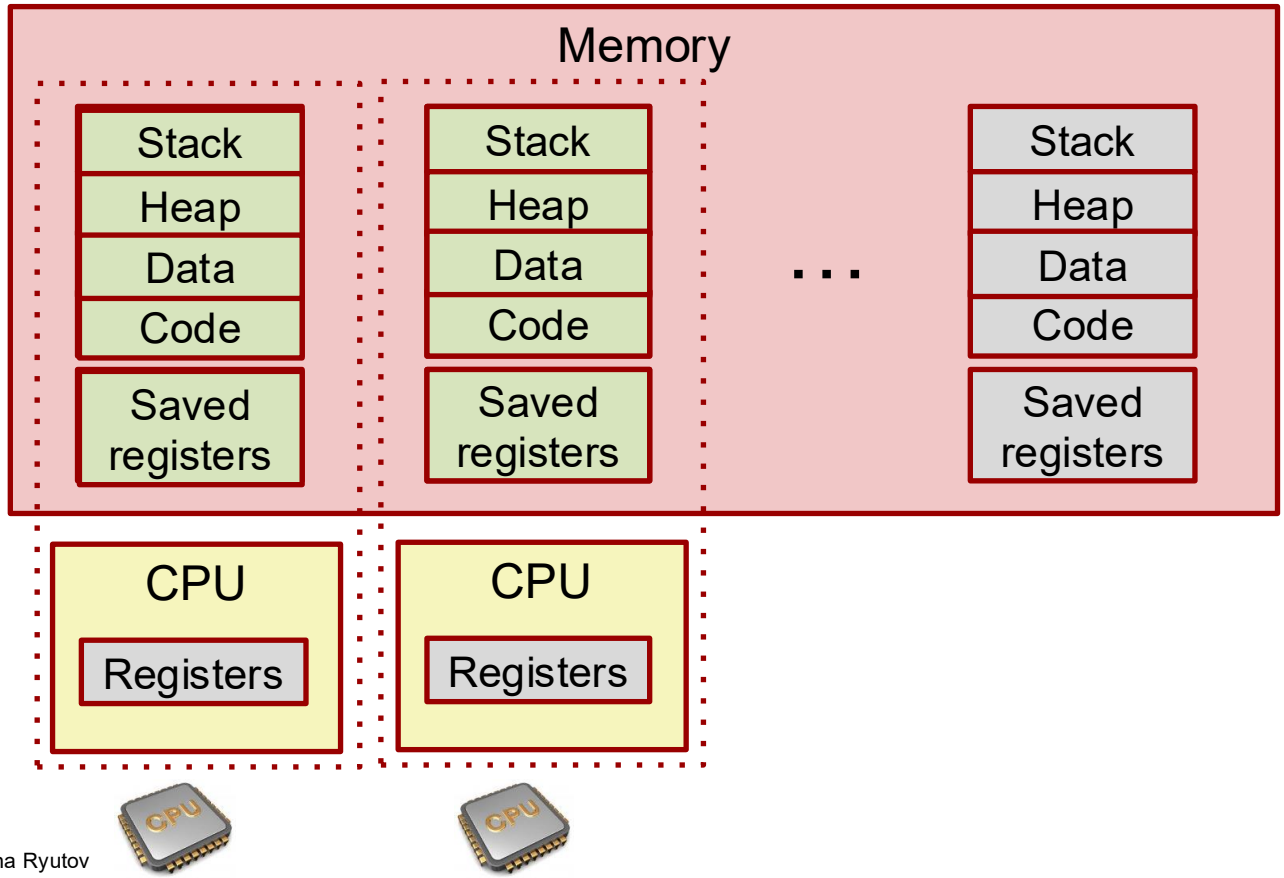


©Tatyana Ryutov

USC Viterbi School of Engineering

The Reality: Multiprogramming

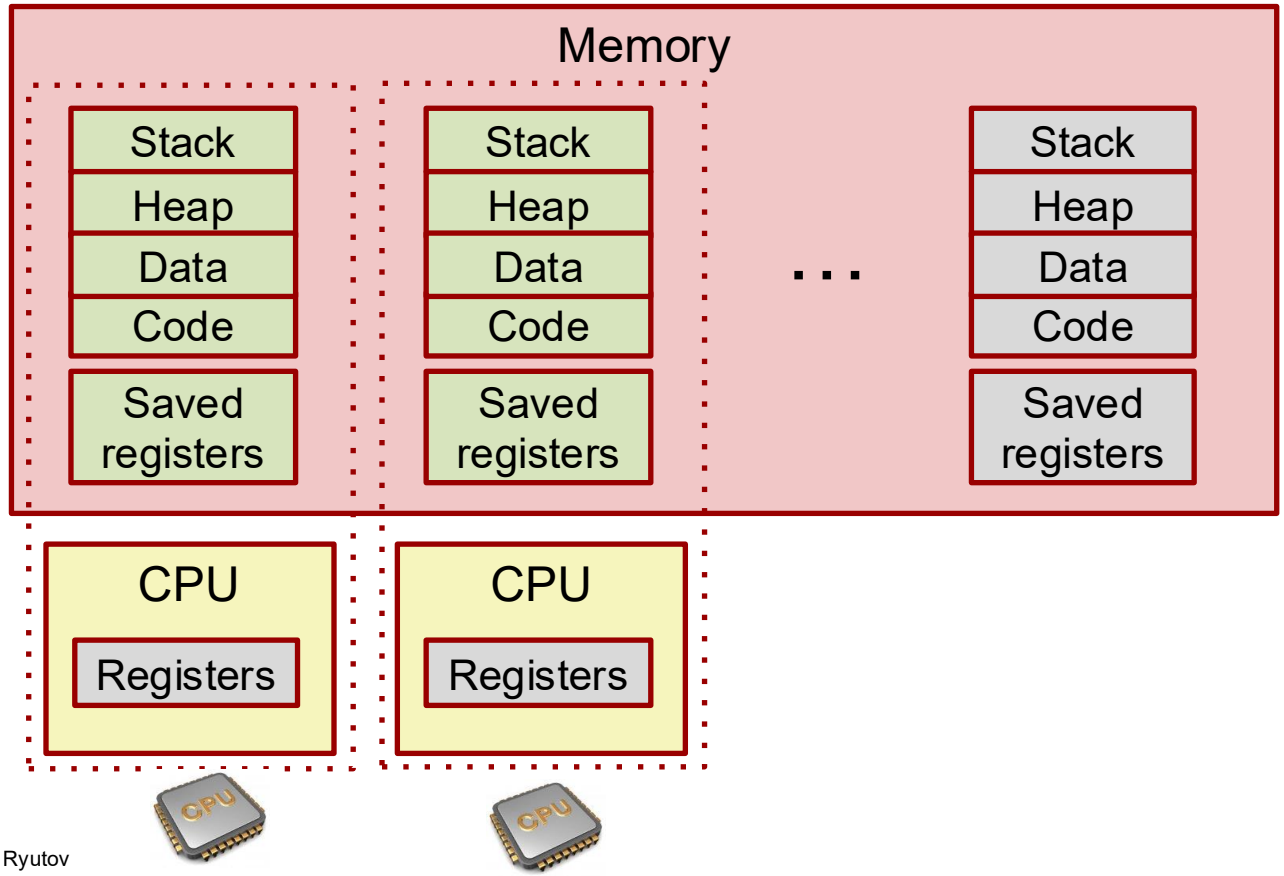
- Single processor executes multiple processes **concurrently**
  - Process executions interleaved, CPU runs one process at a time
  - Address spaces managed by virtual memory system (later in course)
  - Execution context (registers, stack, …) for other processes saved in memory



©Tatyana Ryutov

The Reality: Multiprocessing

- Multicore processors
  - Multiple CPUs ("cores") on a single chip share main memory (and some of the caches)
  - Each can execute a separate process
    - Kernel schedules processes to cores
    - Constantly swapping processes



©Tatyana Ryutov

USC Viterbi School of Engineering

Outline

- Review
- Concurrency
- Thread abstraction
- Thread operations
- Implementing threads
- Synchronization intro

©Tatyana Ryutov

Motivation for Thread Abstraction

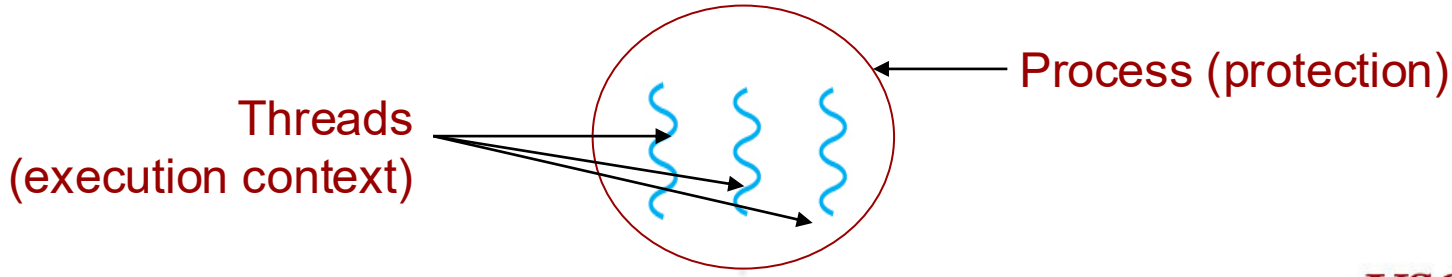
1. Make concurrency cheaper
   - Process overhead
     - Creating a new process is costly
     - Communicating between processes is costly
     - Switching between process is costly
2. Address human limitations
   - OSs (and apps) often need to handle multiple things happening at the same time
   - Humans are not very good at keeping track of multiple things happening simultaneously

- Is there a model that will solve these problems?

©Tatyana Ryutov

Thread Abstraction

- Key ideas: decouple concurrency from protection and provide a familiar programming model
- Thread is an OS abstraction of a CPU as exposed to programs
- Thread is a **single execution sequence** that represents a **separately schedulable** task
  - Single execution sequence: familiar programming model
  - Separately schedulable: OS can run or suspend a thread at any time
  - Bound to a single process
- Threads define a set of tasks that run **concurrently** while the code for each task is **sequential**
- Old-fashioned name for threads - light weight processes

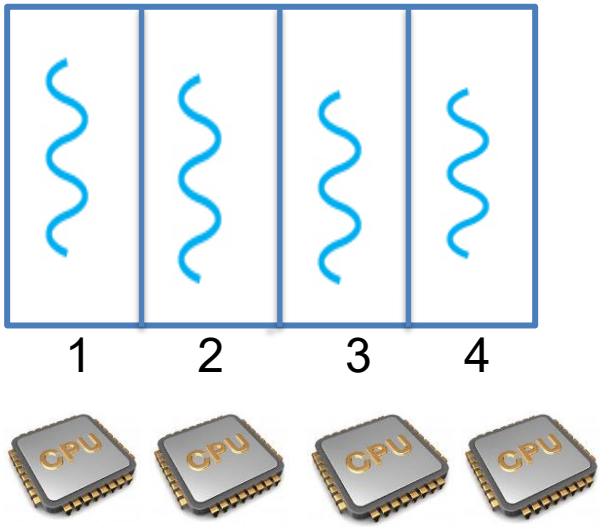


©Tatyana Ryutov

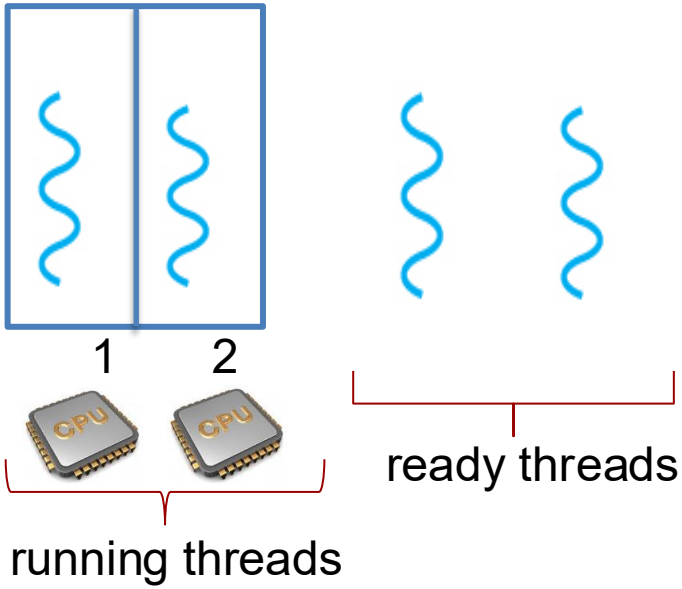
Thread Abstraction Continued

- OS creates an illusion of infinite number of processors
  - Provides virtualization of the processor
    - OS multiplexes threads on the available processors by suspending and resuming threads transparently
    - Number of threads >> number of processors

Programmer abstraction



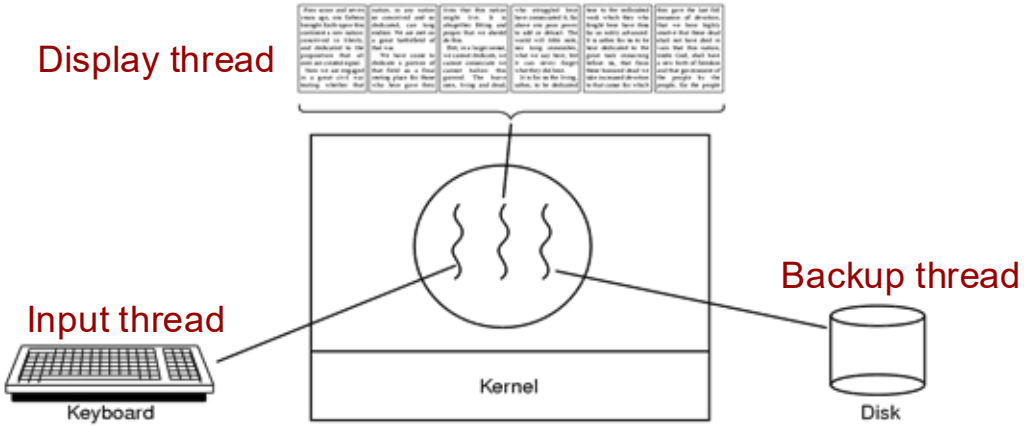
Physical reality



©Tatyana Ryutov

Why Use Threads?

- To enhance parallel processing
  - Take advantage of multiple cores
- To utilize the idle time of the CPU
- Responsiveness
  - E.g., main thread executes while another waits for I/O
- Efficient resource sharing
  - No calls to OS
- Economy
  - Threads are cheap compared to processes



©Tatyana Ryutov

USC Viterbi School of Engineering

Implementing Threads

- Thread support (API) is provided at either:
  - User level → user-level threads
    - Supported and managed **entirely** at the user level
    - User-level library provides thread abstractions, scheduling, synchronization, etc.
  - Kernel level → kernel-level threads
    - Supported and managed **directly** by the OS
    - Thread abstraction only available to kernel
    - The majority of commercial OSs support kernel threads

©Tatyana Ryutov

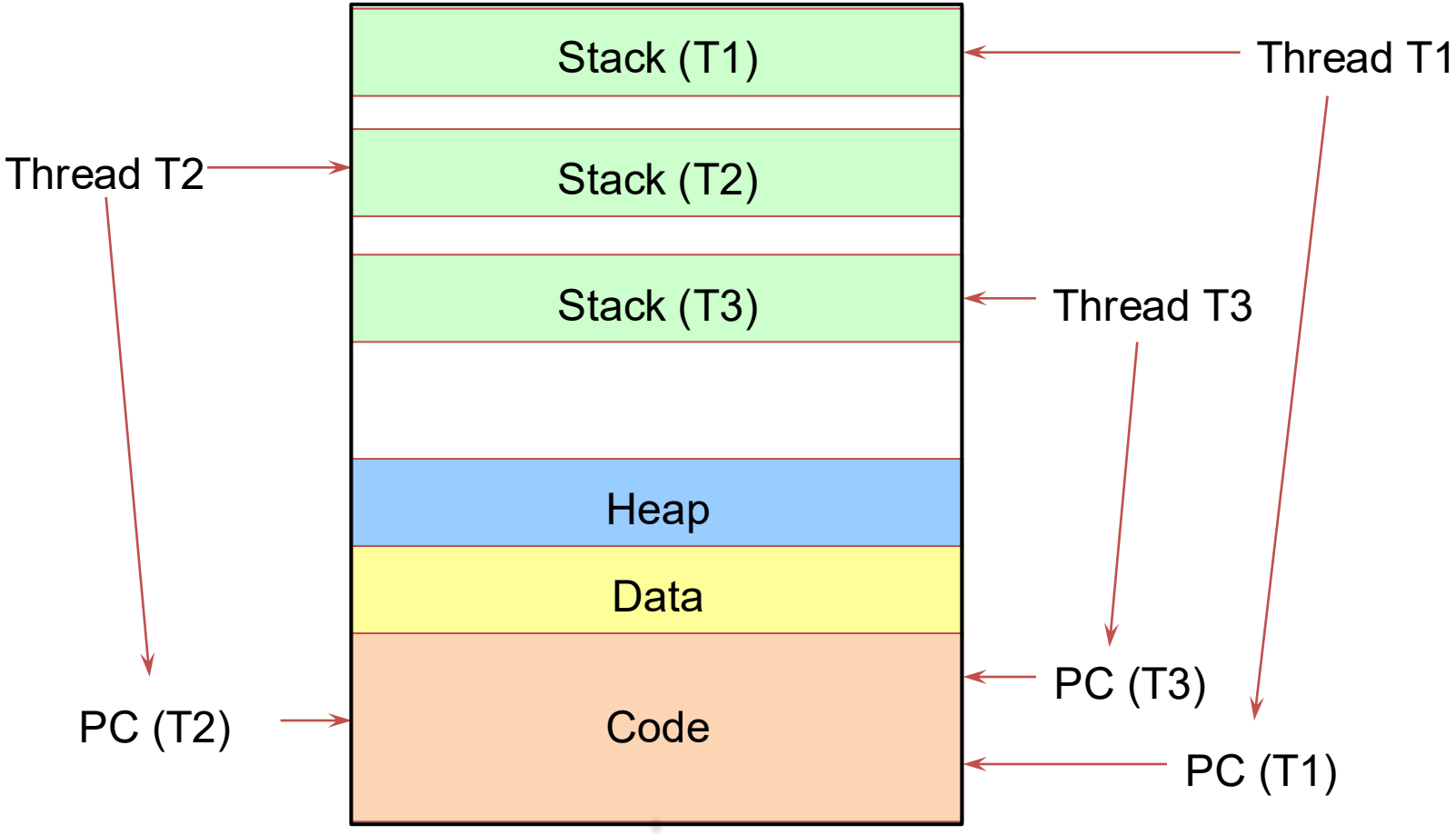
Thread Control Block (TCB)

- TCBs can be maintained by the kernel and stored in some kernel data structure

Thread ID
Thread state
Thread priority
CPU information: • Program counter • Registers • Stack pointer
Pointer to parent process
Pointer(s) to child threads(s)
…

©Tatyana Ryutov

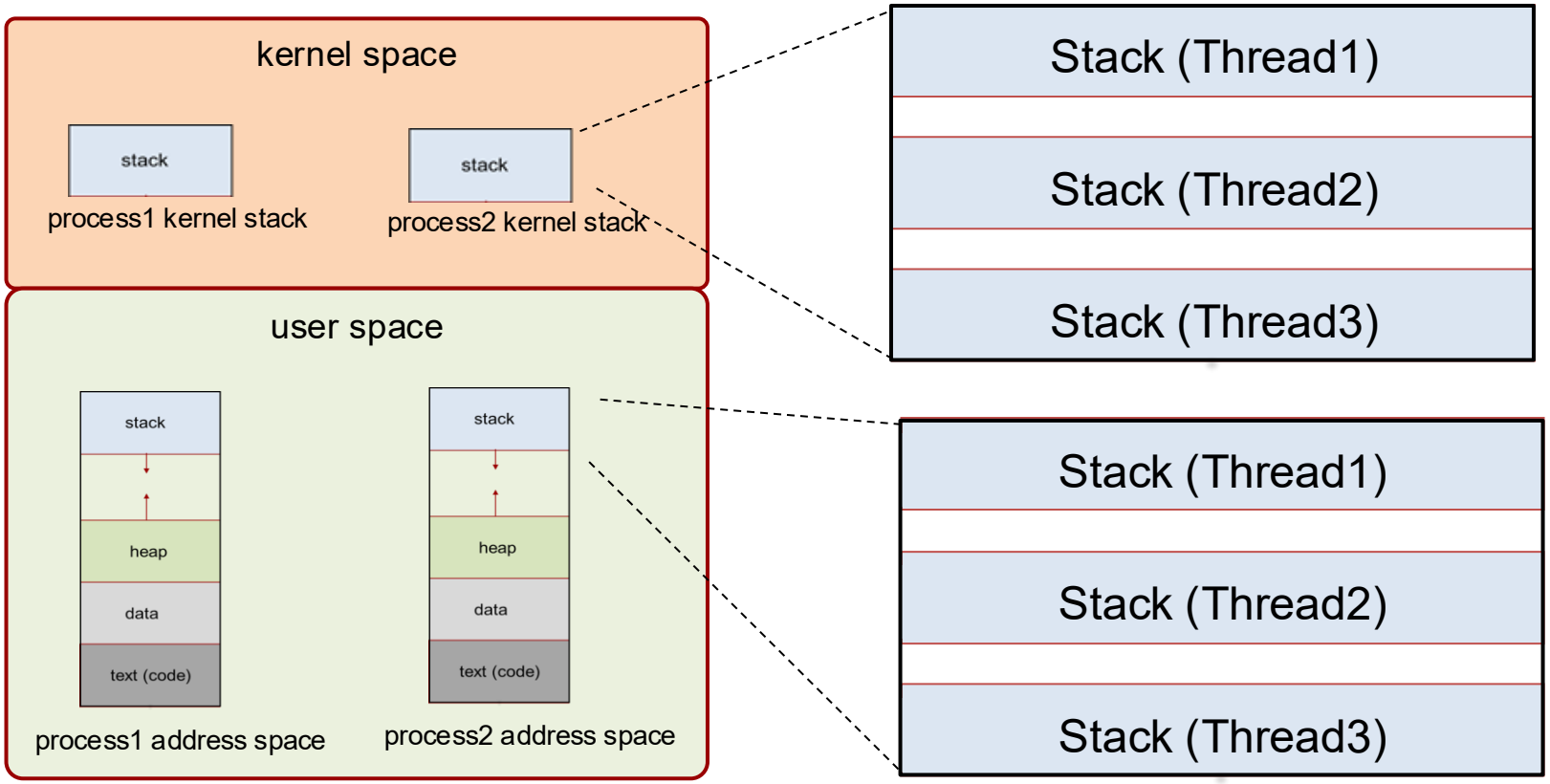
Threads in a Process



process address space

©Tatyana Ryutov

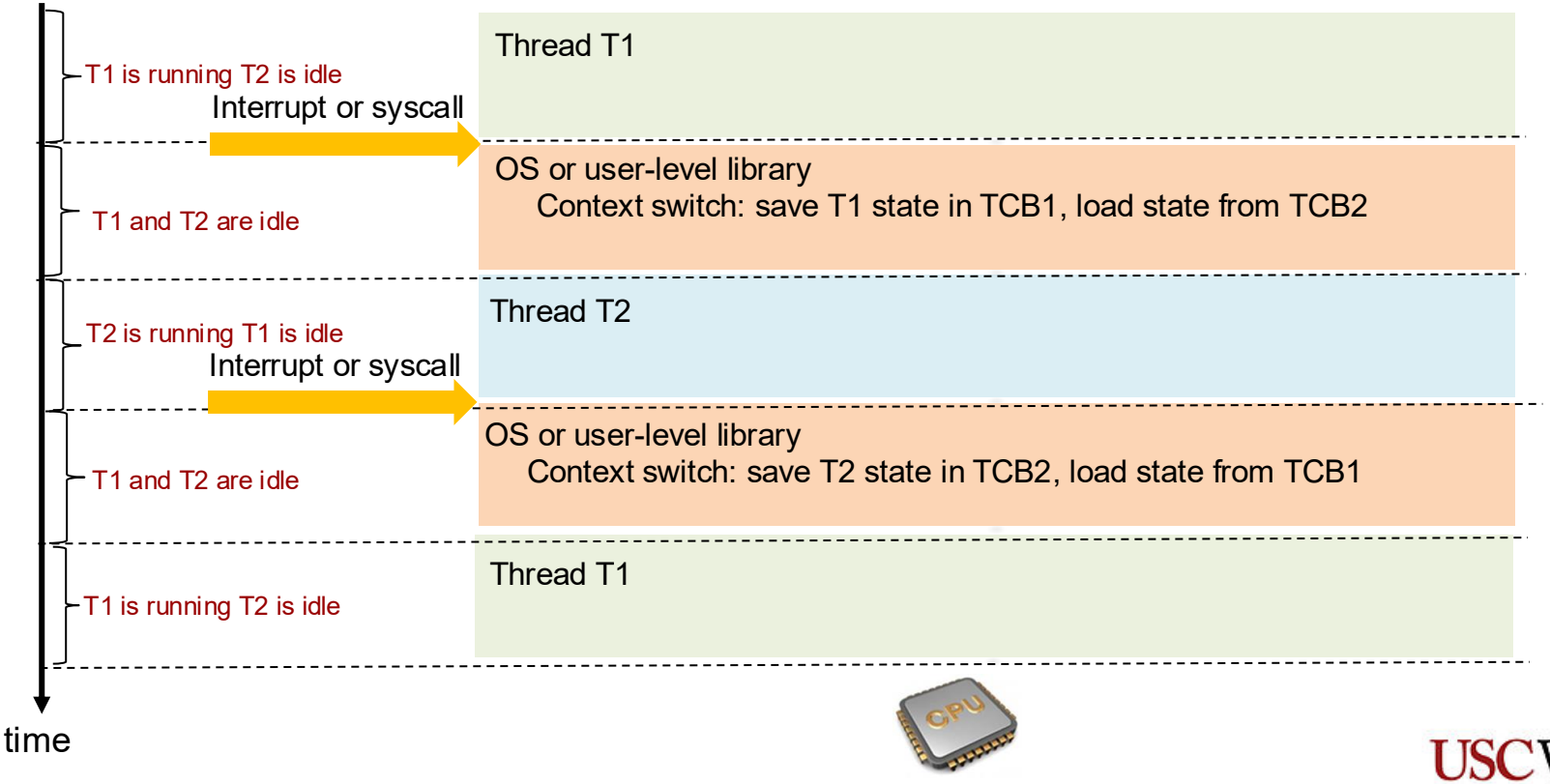
Kernel and User Stack (kernel-level threads)



©Tatyana Ryutov

Thread Context Switch

- Scheduler choses one of the ready threads for execution
  - Different scheduling policies, e.g., FIFO, round robin, etc. (later in class)
- Context switch: swapping threads
  - Save the state (registers + PC) of the previously executing thread then restore the state of the chosen thread

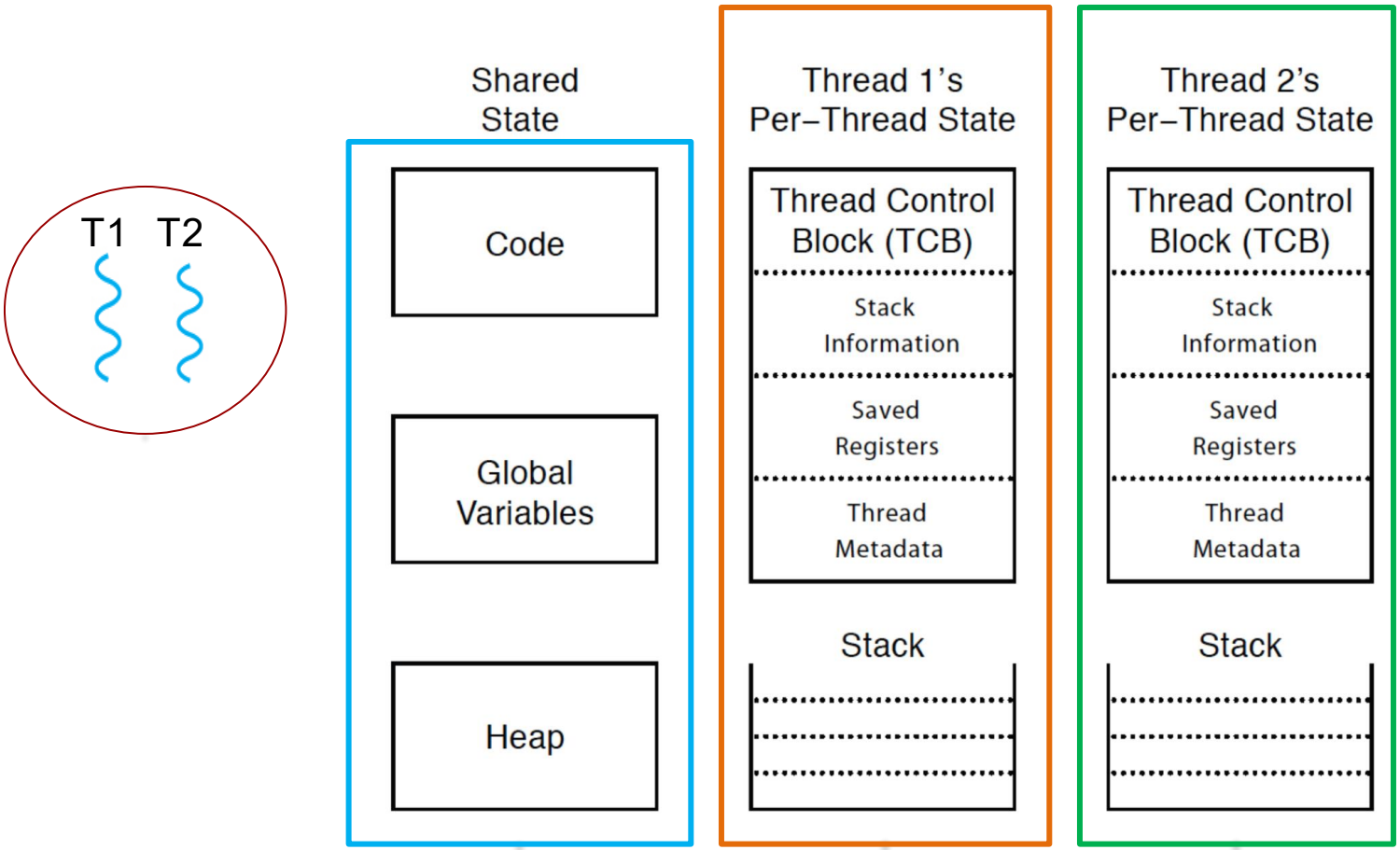


©Tatyana Ryutov

USC Viterbi School of Engineering

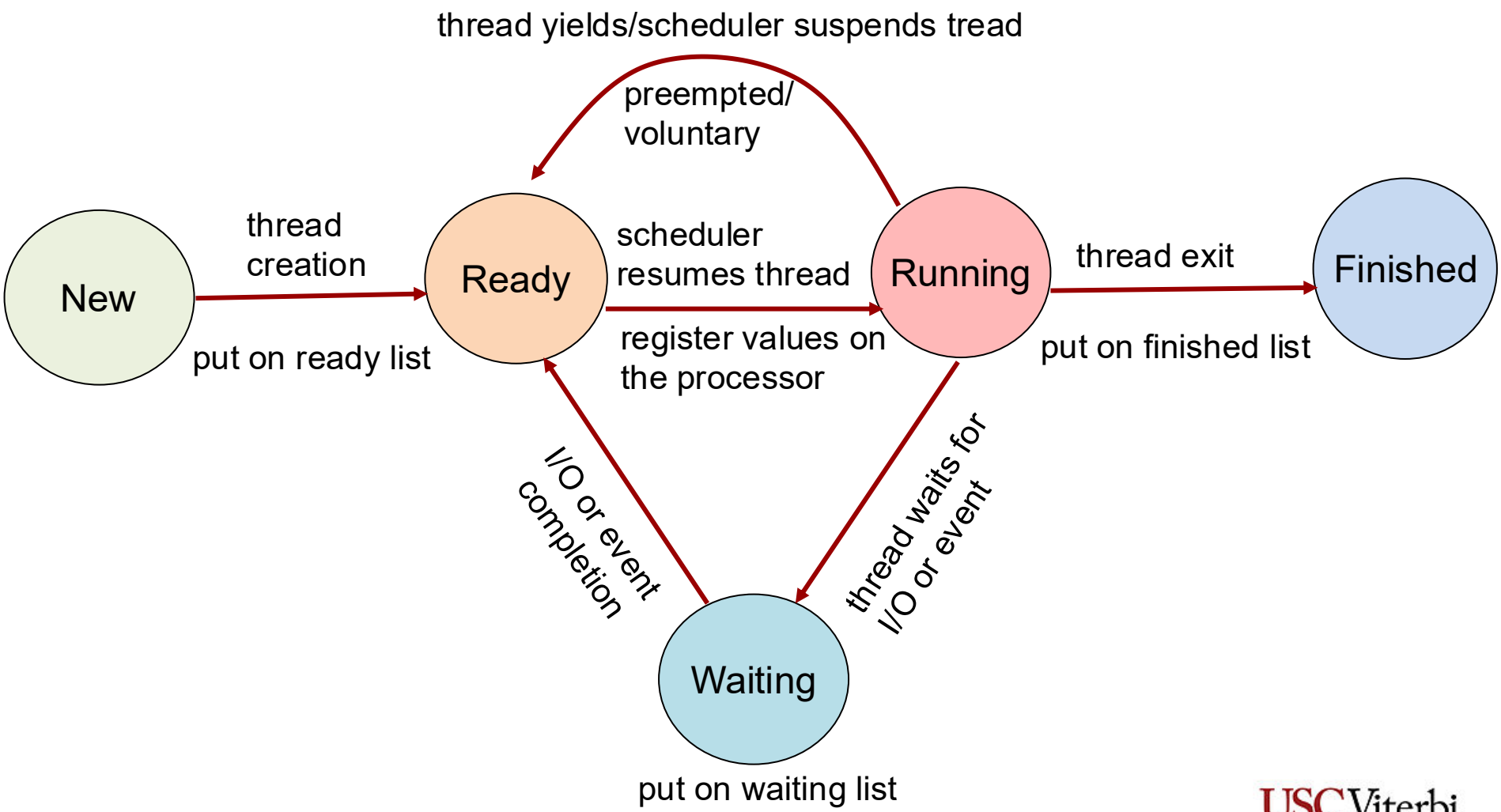
Thread Data Structures

- Thread is cheaply represented by a PC, registers, stack, and small thread control block (TCB)



©Tatyana Ryutov

Thread Lifecycle

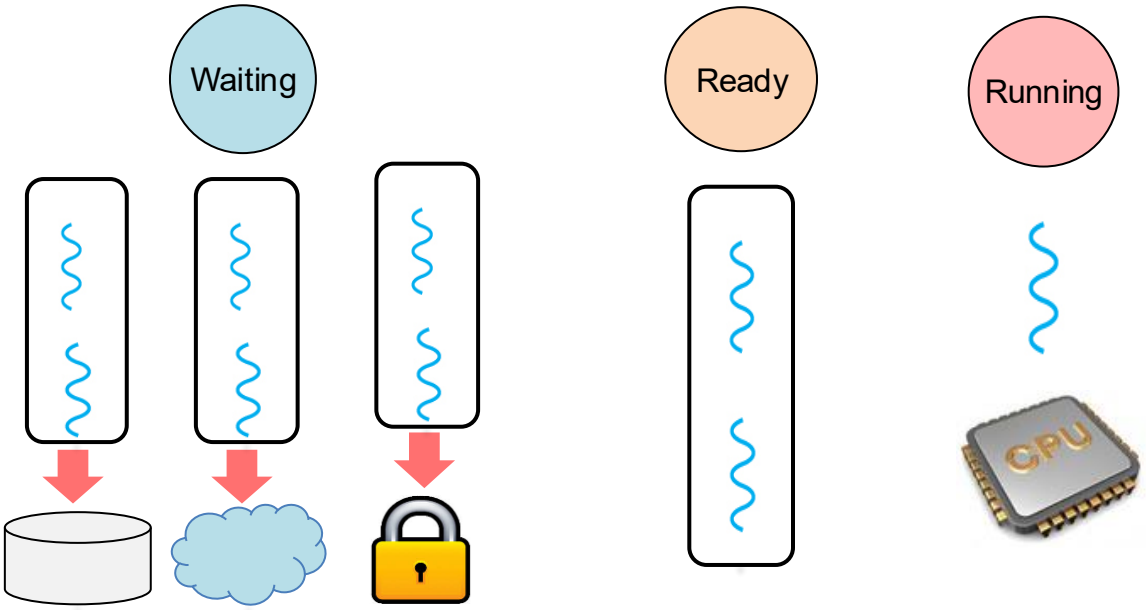


©Tatyana Ryutov

USC Viterbi School of Engineering

Ready and Wait Queues

- Kernel keeps ready threads in one or more Ready queue data structures
- Kernel puts a thread on Wait queue when it blocks, and transfers it to Ready queue when it is ready to run again



©Tatyana Ryutov

USC Viterbi School of Engineering

Outline

- Review
- Concurrency
- Thread abstraction
- Thread operations
- Implementing threads
- Synchronization intro

©Tatyana Ryutov

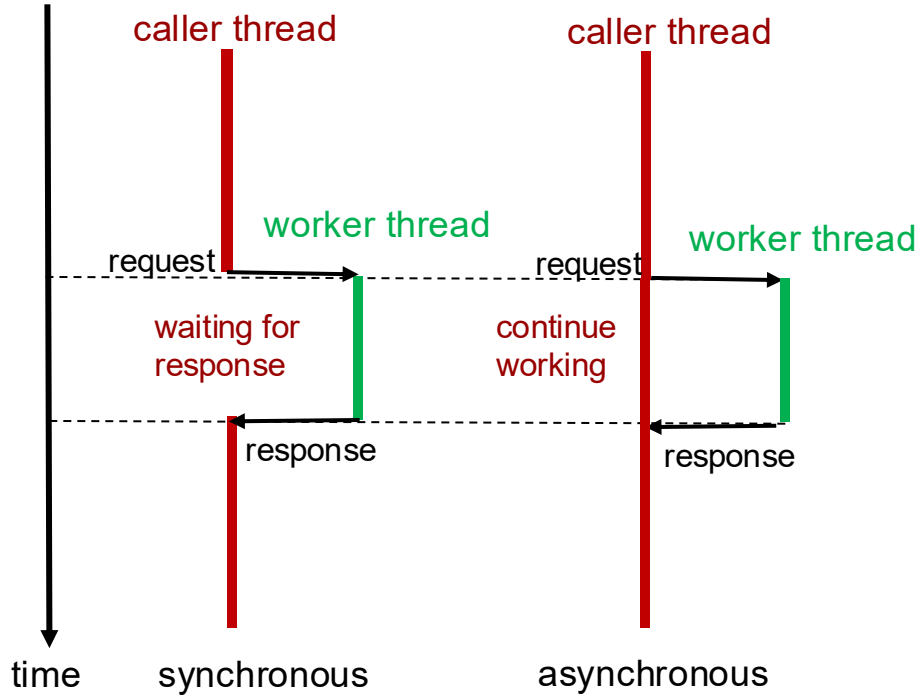
Thread Operations

- Simplified API is based on the POSIX standard pthreads API:
  - **`void thread_create(thread, func, args)`**
    - Create a new thread to run **`func(args)`**
  - **`void thread_yield()`**
    - Give up processor voluntarily
  - **`int thread_join(thread)`**
    - In parent, wait for created thread to exit, then return the value passed to **`thread_exit`** by that thread
  - **`void thread_exit(ret)`**
    - Quit thread and clean up, wake up joiner, if any
    - Store the value **`ret`** in the current thread's data structure

©Tatyana Ryutov

Strategies for Creating Threads

- Synchronous (blocking):
  - Caller is blocked until procedure is completed
  - Typically significant data sharing between threads
- Asynchronous (non-blocking):
  - Caller resumes execution right after **`thread_create`**
  - Can wait for the function completion with **`thread_join`**
  - Typically independent threads, little data sharing



©Tatyana Ryutov

Example: threadHello

```
#define NTHREADS 10
thread_t threads[NTHREADS];

main() {
    // Create 10 threads
    for (i = 0; i < NTHREADS; i++) thread_create(&threads[i], &go,  i);

    for (i = 0; i < NTHREADS; i++) {
        exitValue = thread_join(threads[i]);
        printf("Thread %d returned with %ld\n", i, exitValue);
    }
    printf("Main thread done.\n");
}

void go (int n) {
    printf("Hello from thread %d\n", n);

    thread_exit(100 + n);
}
```

©Tatyana Ryutov

threadHello: Example Output

```
#define NTHREADS 10
thread_t threads[NTHREADS];

main() {
    // Create 10 threads
    for(i = 0; i < NTHREADS; i++)thread_create(&threads[i], &go, i);

    for(i = 0; i < NTHREADS; i++) {
        exitValue = thread_join(threads[i]);
        printf("Thread %d returned with %ld\n", i, exitValue);
    }
    printf("Main thread done.\n");
}

void go (int n) {
    printf("Hello from thread %d\n", n);

    thread_exit(100 + n);
}
```

```
bash-3.2$ ./threadHello
Hello from thread 0
Hello from thread 1
Thread 0 returned 100
Hello from thread 3
Hello from thread 4
Thread 1 returned 101
Hello from thread 5
Hello from thread 2
Hello from thread 6
Hello from thread 8
Hello from thread 7
Hello from thread 9
Thread 2 returned 102
Thread 3 returned 103
Thread 4 returned 104
Thread 5 returned 105
Thread 6 returned 106
Thread 7 returned 107
Thread 8 returned 108
Thread 9 returned 109
Main thread done.
```

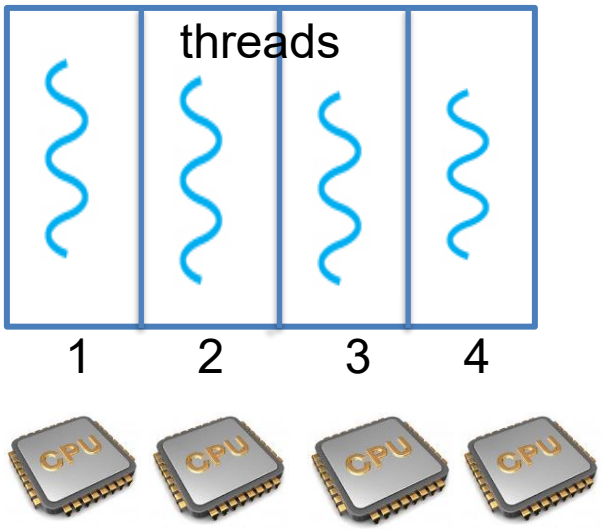
USC Viterbi School of Engineering

©Tatyana Ryutov

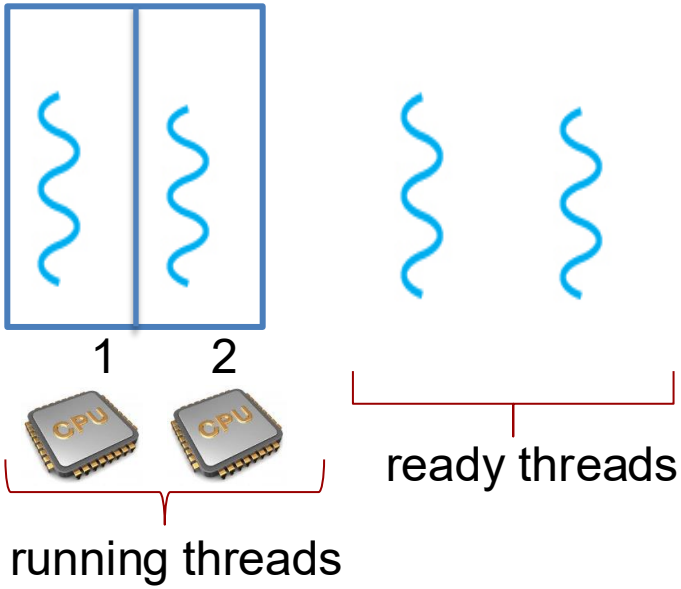
Reasoning about Correctness

- Each thread runs on a dedicated virtual processor with unpredictable and variable speed

Programmer abstraction



Physical reality



©Tatyana Ryutov

USC Viterbi School of Engineering

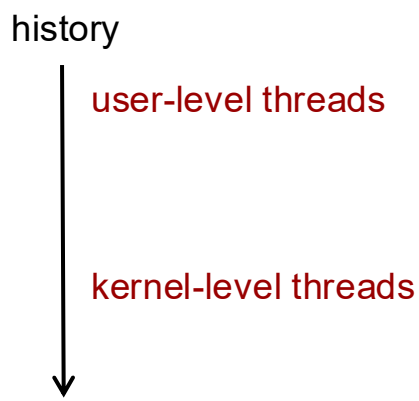
Outline

- Review
- Concurrency
- Thread abstraction
- Thread operations
- Implementing threads
- Synchronization intro

©Tatyana Ryutov

Recall: Implementing Threads

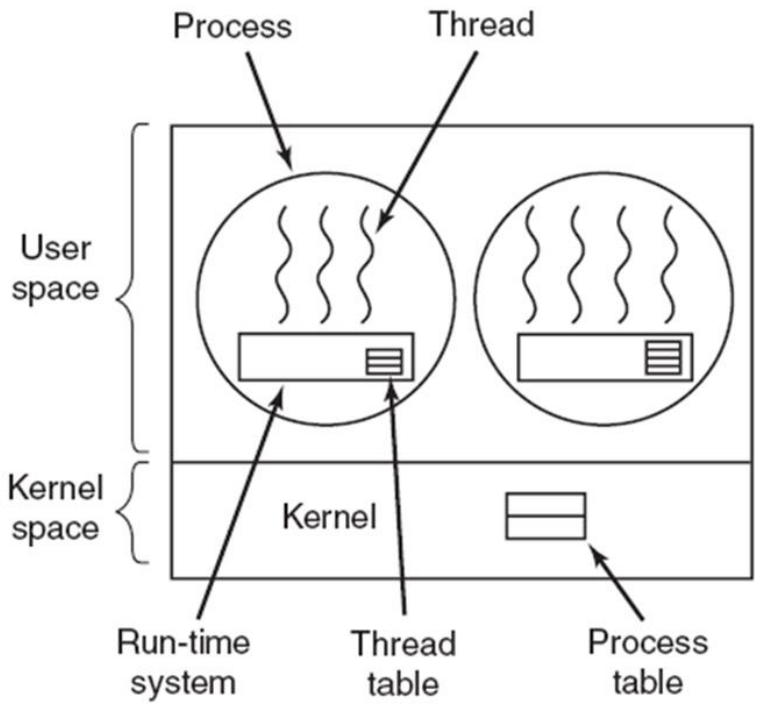
- Thread support (API) is provided at either:
  - User level → user-level threads
    - Supported and managed **entirely** at the user level
    - User-level library provides thread abstractions, scheduling, synchronization, etc.
  - Kernel level → kernel-level threads
    - Supported and managed **directly** by the OS
    - Thread abstraction only available to kernel
    - The majority of commercial OSs support kernel threads



©Tatyana Ryutov

User-Level Threads

- Thread library entirely executed in user mode
  - TCBs, ready list, finished list, waiting lists all are data structures in the user process's address space
  - Calls to the thread library are just procedure calls for thread creation/distraction, scheduling, saving/restoring state, etc.
- To OS, a multi-threaded application appears to be a single-threaded process
  - Running thread is mapped to one kernel thread

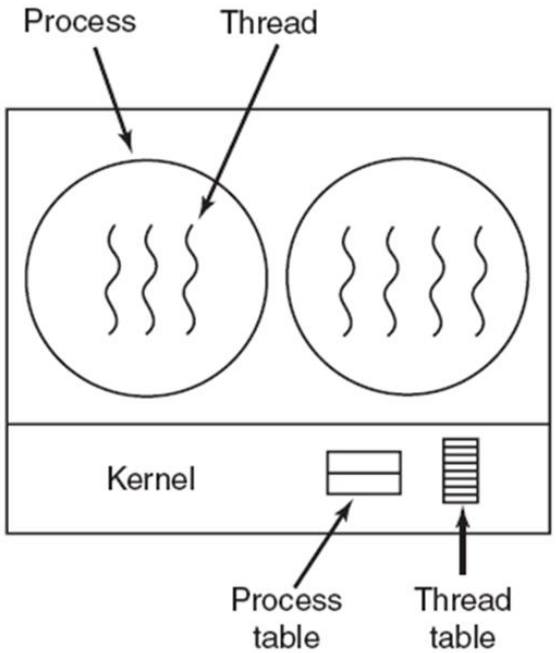


©Tatyana Ryutov

USC Viterbi School of Engineering

Kernel-Level Threads

- OS manages threads and processes
  - One process control block (PCP) per process
  - One thread control block (TCB) per thread
- All thread operations are implemented in the kernel and the OS schedules all threads in the system
- OS provides system calls to create and manage threads from user space (focus of Project2)



©Tatyana Ryutov

USC Viterbi School of Engineering

Tradeoffs

- User-level Threads
  - Advantages:
    - Thread switching does not require kernel involvement
    - Scheduling can be application-specific
    - Can run on any OS
  - Disadvantages:
    - When a thread executes a system call, not only is that thread blocked, but also all of the threads within the process
    - Multithreaded application cannot take advantage of multiprocessing
    - Kernel can make poor process scheduling decisions
- Kernel-level Threads
  - Advantages
    - The kernel can simultaneously schedule multiple threads from the same process on multiple processors
  - Disadvantages
    - The transfer of control from one thread to another within the same process requires switch to kernel

©Tatyana Ryutov

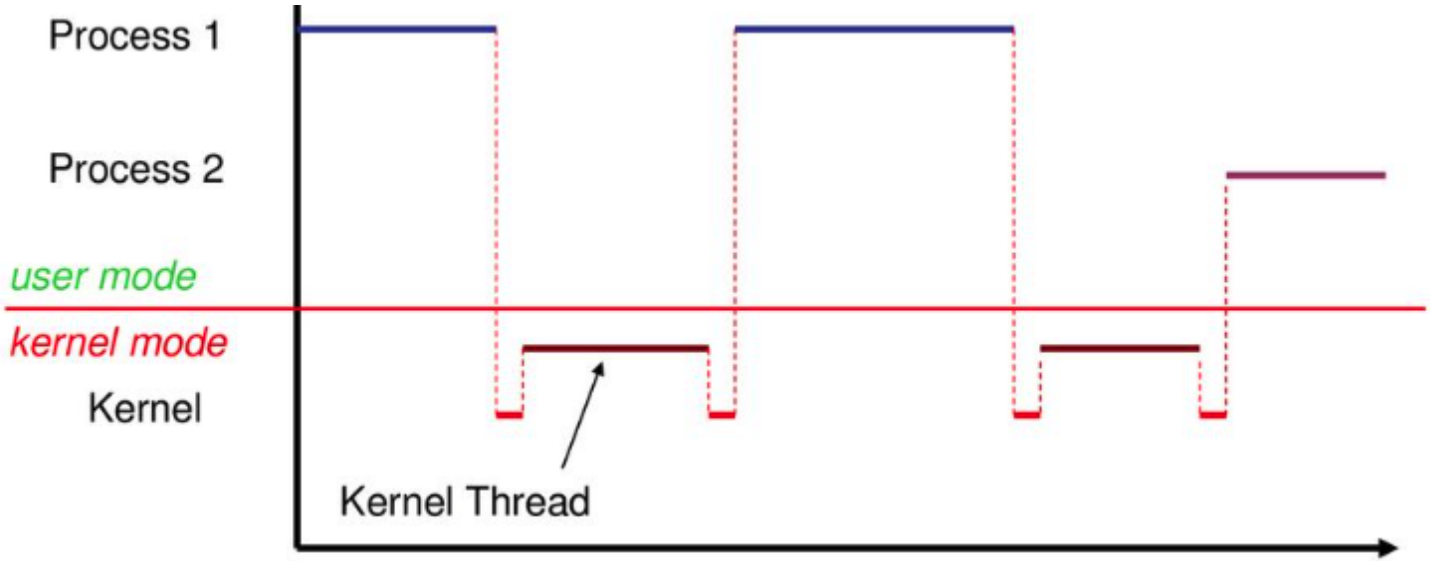
Thread Libraries

- Three main thread libraries in use today:
  - POSIX Pthreads
    - May be provided either as user-level or kernel-level
    - A POSIX standard (IEEE 1003.1c) API for thread creation and synchronization
  - Win32
    - Kernel-level library on Windows system
    - Implements the one-to-one (user to kernel thread) mapping
  - Java
    - Java threads are managed by the JVM
    - Typically implemented using thread model provided by OS

©Tatyana Ryutov

Kernel Treads and Kernel-level Threads

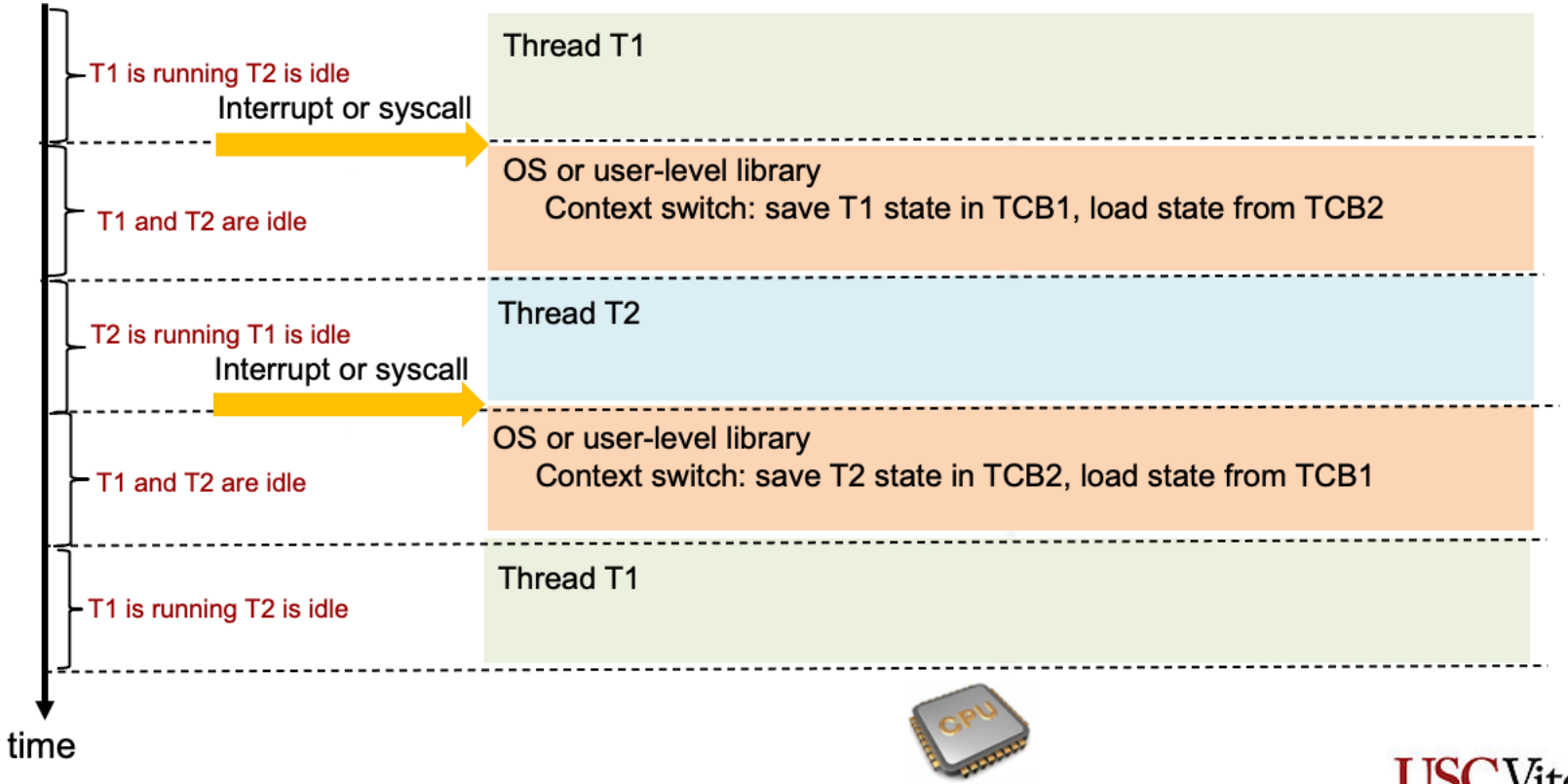
- "Kernel threads" refer to threads that run **entirely** in kernel space (never run in user mode)
  - Most OSs support those
  - Example: kthreads - multi-threading technique done in the kernel
- Not the same as "kernel-level threads"



©Tatyana Ryutov

Recall: Thread Context Switch

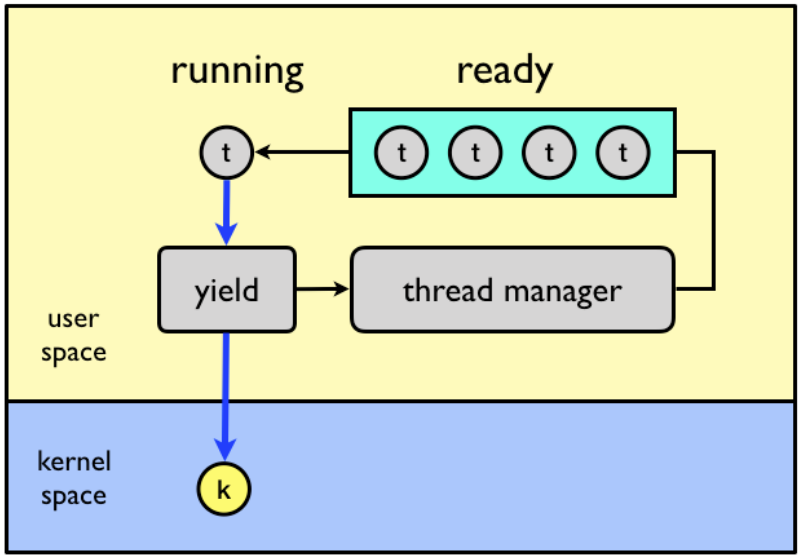
- Context switch: swapping threads
- When to context switch?
  - Cooperative (non-preemptive) multithreading (multitasking) and non-cooperative (preemptive) multithreading (multitasking)



©Tatyana Ryutov

Cooperative Multithreading

- Cooperative threads use non-preemptive scheduling
- Running thread decides when to give up the CPU
  - Thread runs without interruption until it explicitly relinquishes control of the processor, e.g., calls **`thread_yield()`** or **`thread_join`**()
  - Running thread saves its own state before yielding
- Cooperative multithreading does not work in multiprocessing environment
- Used in some older OSs
  - E.g., early versions of Apple's MacOS, Windows 3.1




```
// main program loop
while(1){
     do_thread1();
     do_thread2();
     do_thread3();
     do_thread4();
}
```

©Tatyana Ryutov

Example: Cooperative Multithreading

- Assume that only two threads exist in a system and thread "Ping" executes first
- What is the output of running these two threads?

Ping thread

```
while (1) {

printf("ping\n");
    thread_yield();
}
```

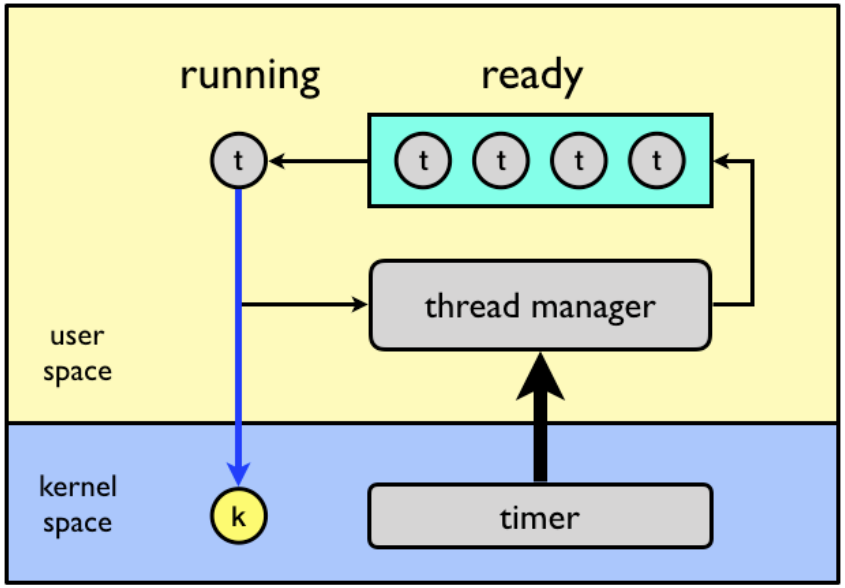
Pong thread

```
while (1) {

printf("pong\n");
    thread_yield();
}
```

©Tatyana Ryutov

Non-Cooperative Multithreading

- A thread can be pre-empted at any point
- Rely on a scheduler to decide which thread to run
  - Generally done based on some regular timer interval, e.g., every 10ms
- Often called pre-emptive multithreading
- Used in most OSs




```
// main program loop
while(1){
     do_thread1_for10ms();
     do_thread2_for10ms();
     do_thread3_for10ms();
     do_thread4_for10ms();
}
```

©Tatyana Ryutov

Tradeoffs

- Cooperative Multithreading
  - Advantages:
    - Increased control over thread interleaving
  - Disadvantages:
    - Badly written code can freeze the system
    - Some tasks can starve
    - Does not work for multi-processor architectures
- Non-cooperative Multithreading
  - Advantages:
    - Badly written code will not freeze the system
    - Better system responsiveness
    - Programs do not have to be written to stop in the middle
  - Disadvantages:
    - More heavyweight

history

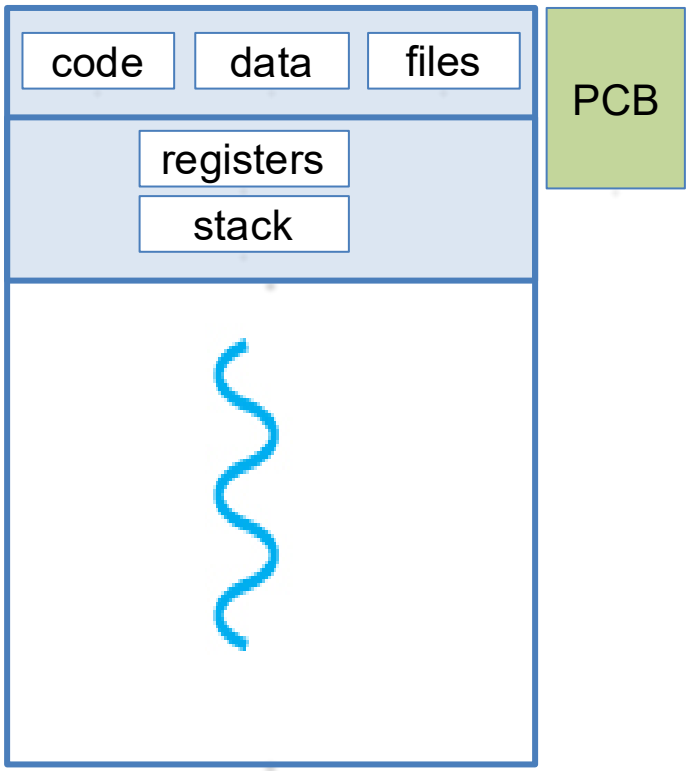
cooperative multithreading

non-cooperative multithreading

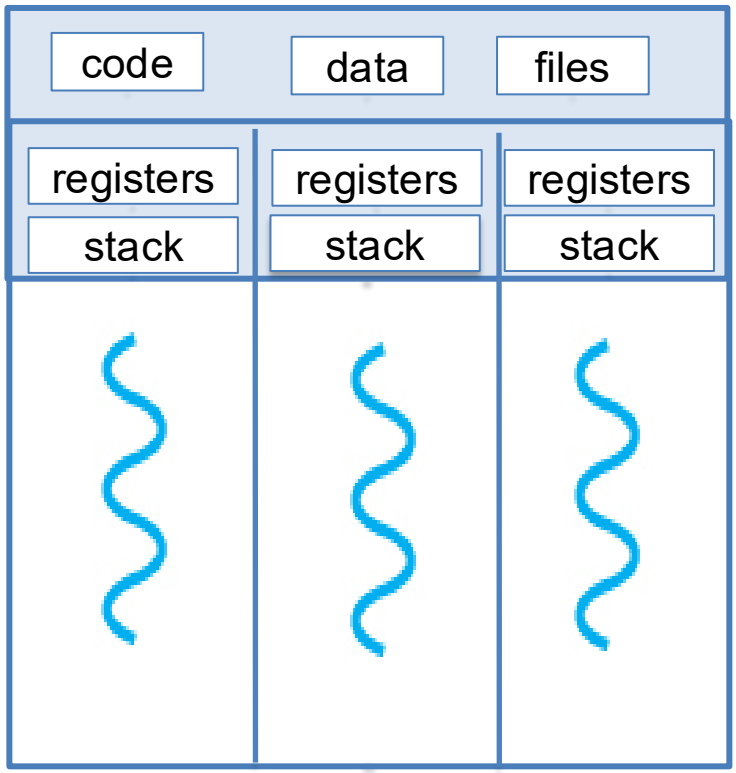
©Tatyana Ryutov

Single and Multi-threaded Processes

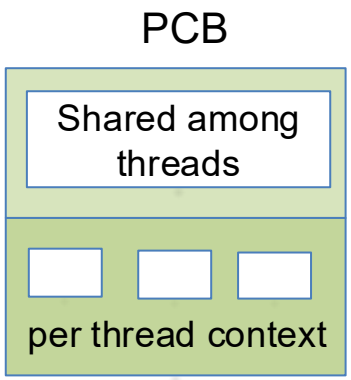
- Process: address space + **threads** + restricted permissions



Single-threaded process



Multithreaded process



©Tatyana Ryutov

USC Viterbi School of Engineering

Threads vs. Processes

Thread	Process
• A thread cannot live on its own, it must live within a process • There can be more than one thread in a process • Threads within a process share code/data/heap, share I/O, but each has its own stack & registers • If a thread finished, its stack is reclaimed • Inter-thread communication via shared memory • Each thread can run on a different physical processor • Inexpensive creation and context switch	• There must be at least one thread in a process • Processes do not share code/data/heap, share • If a process finished, its resources are reclaimed and all threads terminated • Inter-process communication via OS • Each process can run on a different physical processor • Expensive creation and context switch

©Tatyana Ryutov

Summary

- Processes are too heavyweight for multiprocessing
- Solution is to separate threads from processes
  - Kernel-level threads much better, but still significant overhead
  - User-level threads even better, but not well integrated with OS
- Scheduling of threads can be either preemptive or non-preemptive
- Processes consist of threads (concurrency) and address space (protection)
- Concurrency accomplished by multiplexing CPU time
- How do we get our threads to correctly cooperate with each other?
  - Synchronization…

©Tatyana Ryutov

Outline

- Review
- Concurrency
- Thread abstraction
- Thread operations
- Implementing threads
- Synchronization intro

©Tatyana Ryutov

Synchronization Challenges

- Program execution depends on the interleaving of threads' access to the shared state
  - Multiprocessing
    - In a system with multiple CPUs, cooperating threads can execute simultaneously
  - Multiprogramming
    - Thread execution can be interleaved because of time-slicing
- Thread schedule is non-deterministic
  - Behavior can change when re-run program
- Compilers and processor HW can reorder instructions
  - Visible when multiple threads interact through shared variables
- Multi-word operations are not atomic

©Tatyana Ryutov

Possible Interleavings

Program

```
partA
partB
```

Thread 1

```
PartA1
PartB1
```

Thread 2

```
PartA2
PartB2
```

- What are the possible interleavings?

1	$PartA_1$	$PartB_1$	$PartA_2$	$PartB_2$
2	$PartA_2$	$PartB_2$	$PartA_1$	$PartB_1$
3	$PartA_1$	$PartA_2$	$PartB_1$	$PartB_2$
4	$PartA_2$	$PartA_1$	$PartB_2$	$PartB_1$
5	$PartA_1$	$PartA_2$	$PartB_2$	$PartB_1$
6	$PartA_2$	$PartA_1$	$PartB_1$	$PartB_2$

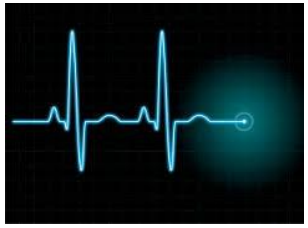
- Execution speeds for different threads can vary from run to run even on the same HW
- How threads are scheduled affects thread's interleavings with other threads
- Make NO assumptions about the relative speed with which different threads execute!

USC Viterbi School of Engineering

©Tatyana Ryutov

Properties of Concurrent Programs (Lamport 1977)

- Properties of concurrent programs are usually divided into two categories:
  - Safety: nothing "bad" happens during execution
    - E.g., a program never terminates with incorrect result
  - Liveness: something "good" eventually happens during the program execution (but we don't know when)
    - E.g., a program eventually terminates
- To prove correctness of a program, one must prove the two properties (necessary, but not sufficient)



©Tatyana Ryutov

Too Much Milk: Solution 1

- Two properties:
  - Safety: Only one thread will get milk
  - Liveness: Some thread should get the milk if needed
- Basic idea of solution 1
  - Leave a note (set a flag)
  - Remove the note (unset the flag)
  - Don't get milk if the note is around

```
if (milk == 0) {
   if (note == 0) {
   note=1;  //leave a note
   milk++;  //get milk
   note=0;  //remove the note
   }
}
```

©Tatyana Ryutov

Solution 1: Safety Violation

time

Thread Jim

```
if (milk == 0) {
```

Context switch

```
        if (note == 0) {
        note=1;  //leave a note;
        milk++;  //get milk;
        note=0;  //remove the note;
        }
}
```

Thread Jeff

```
if (milk == 0) {
        if (note == 0) {
        note=1; //leave note
        milk++; //get milk
        note=0; //remove note
        }
}
```

©Tatyana Ryutov

Too Much Milk, Solution 2

Thread Jim

```
noteJim=1; //leave note
if (noteJeff == 0) {
    if (milk == 0)
        milk++;
}
noteJim=0;
```

Thread Jeff

```
noteJeff=1; //leave note
if (noteJim == 0) {
    if (milk == 0)
        milk++;
}
noteJeff=0;
```

Idea: Use different notes to avoid the possibility of one thread making a decision to buy milk before notifying the other thread

©Tatyana Ryutov

Too Much Milk, Solution 2

Thread Jim

```
noteJim=1; //leave note
```

Context switch

```
if (noteJeff == 0) {
    if (milk == 0)
        milk++;
}
noteJim=0;
```

Thread Jeff

```
noteJeff=1; //leave note
```

Context switch

```
if (noteJim == 0) {
    if (milk == 0)
        milk++;
}
noteJeff=0;
```

©Tatyana Ryutov

Too Much Milk, Solution 3

```
Thread Jim

noteJim=1;
while (noteJeff == 1){ //X
    ; //spin
}
if (milk == 0)
     milk++;
noteJim=0;
```

```
Thread Jeff

noteJeff=1;
if (noteJim == 0){ //Y
    if (milk == 0)
        milk++;
}
noteJeff=0;
```

Can guarantee at points X and Y that either:

(i) Safe for me to buy

(ii) Other will buy, ok to quit

©Tatyana Ryutov

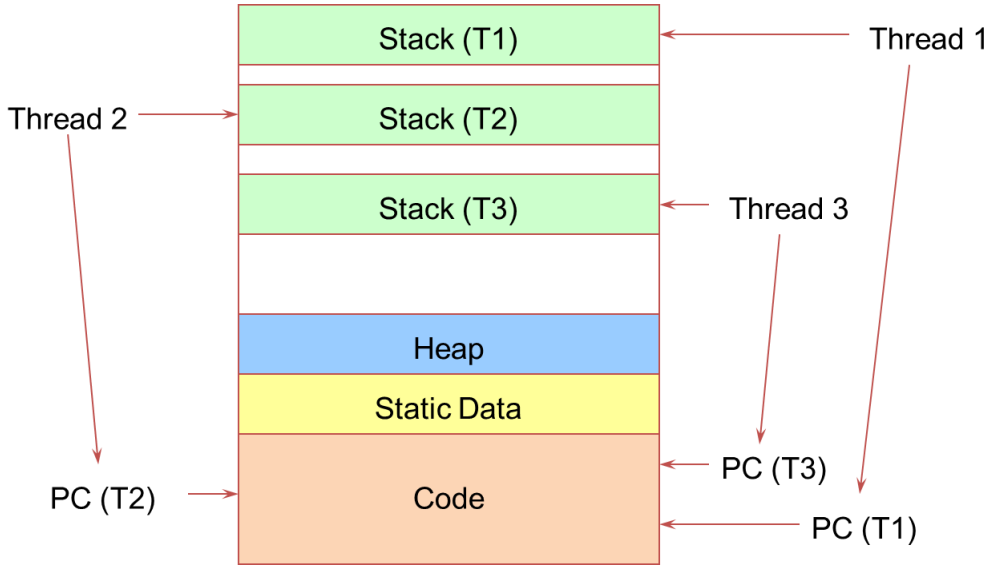
Lessons Learned

- Although it works (assumption: processor execute instructions in program order), solution 3 is complicated:
  - Difficult to verify correctness
  - Two threads have different code
  - While thread is waiting, it consumes CPU time (busy waiting)
- How can we do better?
  - Use HW features to eliminate busy waiting
  - Define higher-level programming abstractions to simplify concurrent programming

©Tatyana Ryutov

Recall: Threads are made to share

- Threads cooperate in multithreaded programs
- Global variables and static objects are shared
- Dynamic objects and other heap objects are shared
- Local variables are not shared
  - Refer to data on the stack
  - Each thread has its own stack



©Tatyana Ryutov

USC Viterbi School of Engineering

Race Condition

- Race Condition: output of a concurrent program depends on the order of operations between threads
  - Occurs when two (or more) threads have conflicting accesses to:
    - Memory location, HW device, file or other OS-level resource
- Goal: ensure that concurrent program works under ALL possible interleaving
- Maintaining data consistency requires **synchronization mechanisms** to ensure the orderly execution:
  - Program behavior to be a specific **function of input** – not who went first
  - Program behavior to be **deterministic** – not to vary from run to run

©Tatyana Ryutov

Race Condition: Example

- Two threads A and B have access to a shared variable “Balance”:

Thread A:
Balance = Balance - 100

Thread B:
Balance = Balance - 200

- Assume that threads A and B are executing concurrently

©Tatyana Ryutov

Race Condition: Example

- The statement " Balance = Balance – 100" is implemented by several machine level instructions such as:

```
LOAD R1, BALANCE    // load Balance from memory into Register 1(R1)
SUB R1, 100         // Subtract 100 from R1
STORE BALANCE, R1   // Store R1's contents back to the memory location
```

- Similarly, "Balance = Balance – 200" can be implemented by the following:

```
LOAD R1, BALANCE
SUB R1, 200
STORE BALANCE, R1
```

Atomic operations: cannot be interleaved with or split by other operations

©Tatyana Ryutov

Race Condition: Example

Scenario 1:

```
LOAD R1, BALANCE
SUB R1, 100
STORE BALANCE, R1
```

Thread A

Context Switch!

```
LOAD R1, BALANCE
SUB R1, 200
STORE BALANCE, R1
```

Thread B

Balance is decreased by 300!

Scenario 2:

```
LOAD R1, BALANCE
SUB R1, 100
```

Thread A

Context Switch!

```
LOAD R1, BALANCE
SUB R1, 200
STORE BALANCE, R1
```

Thread B

lost

Context Switch!

```
STORE BALANCE, R1
```

Thread A

Balance is decreased by 100!

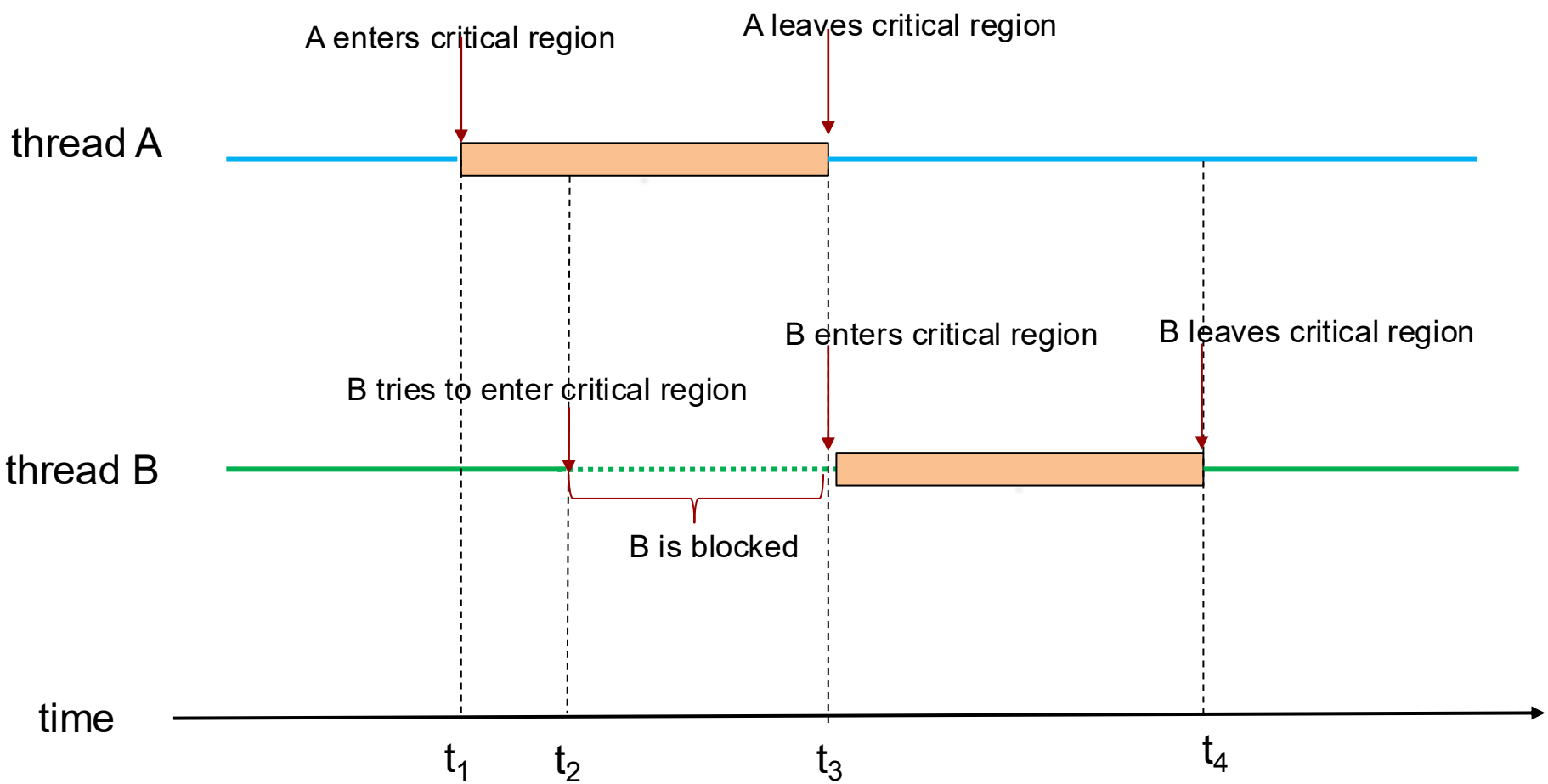
©Tatyana Ryutov

Definitions

- Synchronization: use atomic (all or nothing) operations to ensure cooperation among threads and avoidance of race conditions
- Mutual exclusion: shared resource can only be accessed by one thread at a time
- Critical section (critical region):
  - Code sequence that is vulnerable to races
  - Every race involves two or more threads executing related critical sections concurrently
- To avoid race conditions ensure **mutual exclusion in critical sections**

©Tatyana Ryutov

Mutual Exclusion using Critical Regions



©Tatyana Ryutov

USC Viterbi School of Engineering

General Structure for Critical Sections

- Every thread has the same pattern:
  - **Entry section**: code to attempt entry to critical section, request "permission"
  - **Critical section**: code that requires isolation (e.g., with mutual exclusion)
  - **Exit section**: cleanup code after execution of critical region
  - **Non-critical section**: everything else

```
...
entry section
critical section
exit section
non-critical section
...
```

- There can be multiple critical sections in a program
  - Only critical regions that access the same shared resource (e.g., data structure) need to synchronize with each other

©Tatyana Ryutov

Solving Critical-Section Problem

- A correct solution to the critical section problem must satisfy:
  - Safety:
    - Nothing bad ever happens!
    - At most one thread may be executing in the critical section (mutual exclusion)
  - Liveness:
    - Eventually something good happens!
    - If one or more threads are in their entry section, then eventually at least one of them enters the critical section
  - Further requirement (making liveness practical):
    - Bounded waiting:
      - No thread should wait forever to enter a critical section

©Tatyana Ryutov